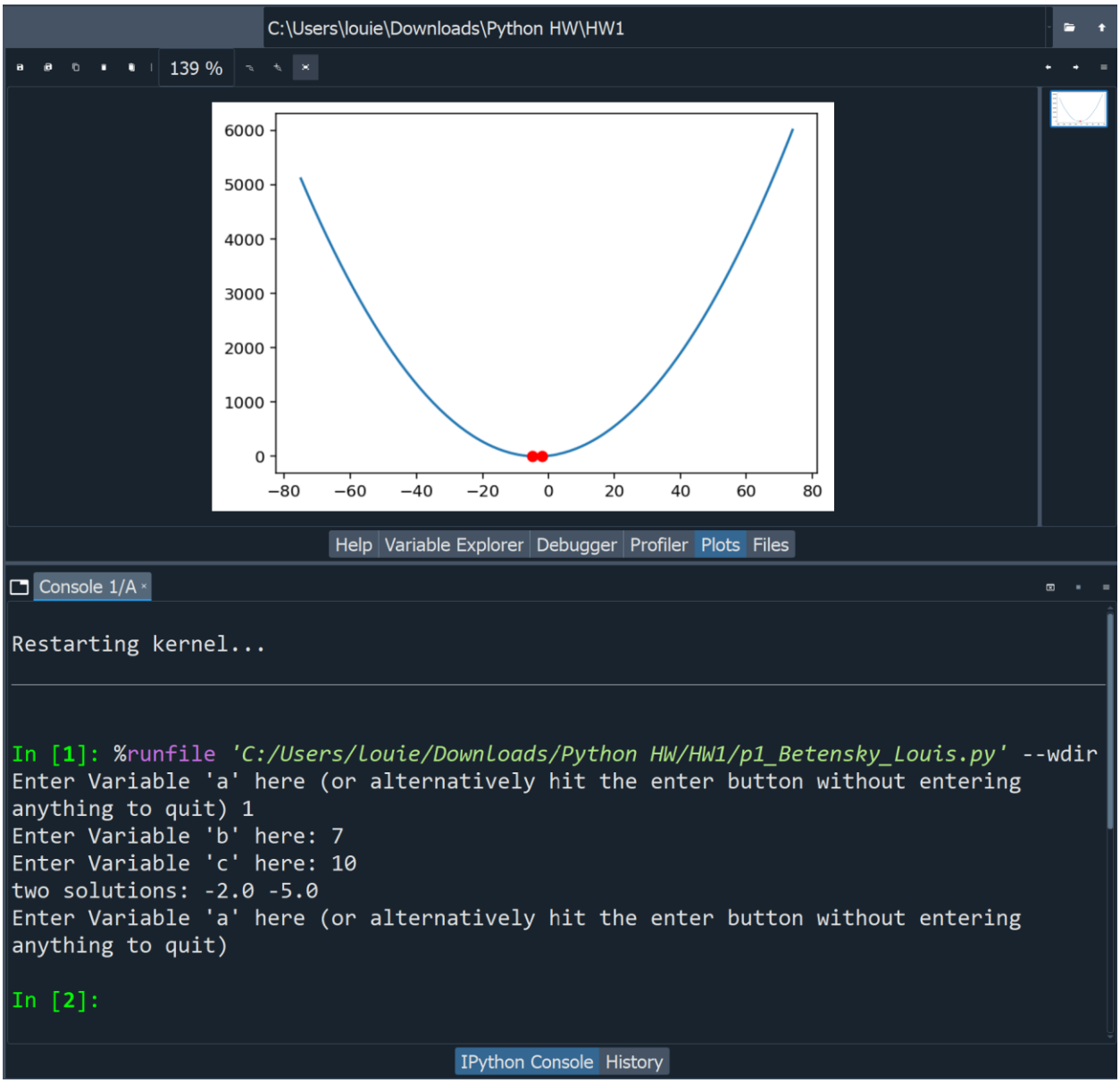
Louis Betensky

Question 1:

```
import math
import matplotlib.pyplot as pylab

stop = False

while not stop:
  #Need to add the exit strat for pressing enter
  a = input("Enter Variable 'a' here (or alternatively hit the enter button without entering anything to quit) ")

  if a == "":
    stop = True
    break

  b = input("Enter Variable 'b' here: ")
  c = input("Enter Variable 'c' here: ")

  roots = 0

  a = float(a)
  b = float(b)
  c = float(c)

  comp = ((b * b) - 4 * a * c)

  if comp < 0:
    print("no real solution")
    roots = 0

  elif comp > 0:
    root1 = (-b + math.sqrt(comp)) / (2 * a)
    root2 = (-b - math.sqrt(comp)) / (2 * a)
    print("two solutions:", root1, root2)
    roots = 2


  else:
    root = (-b + math.sqrt(comp)) / (2 * a)
    print("one solution:", root)
    roots = 1

  xvalues = []
  yvalues = []
  for x in range(-75, 75): #Create a range of 150 points, evenly distributed between negative and positive numbers
    xvalues.append(x)
    y = (a * (x ** 2)) + (b * x) + c
    yvalues.append(y)

  #PDF says we need to show the values on the graph, but i think it might also be asking for showing all points
  #So the first uncommented below is for just the line, and the commented out one shows all points in blue
  pylab.plot(xvalues, yvalues)
  #pylab.plot(xvalues, yvalues, "bo")
  if roots == 2:
    pylab.plot(root1, 0, 'ro')
    pylab.plot(root2, 0, 'ro')
  elif roots == 1:
    pylab.plot(root, 0, 'ro')
  else:
    center = -b / (2 * a)
    pylab.plot(center, 0, 'ro')
    print("Setting center to domains max / minimum: " + str(center))

  pylab.plot()
  pylab.show()
```

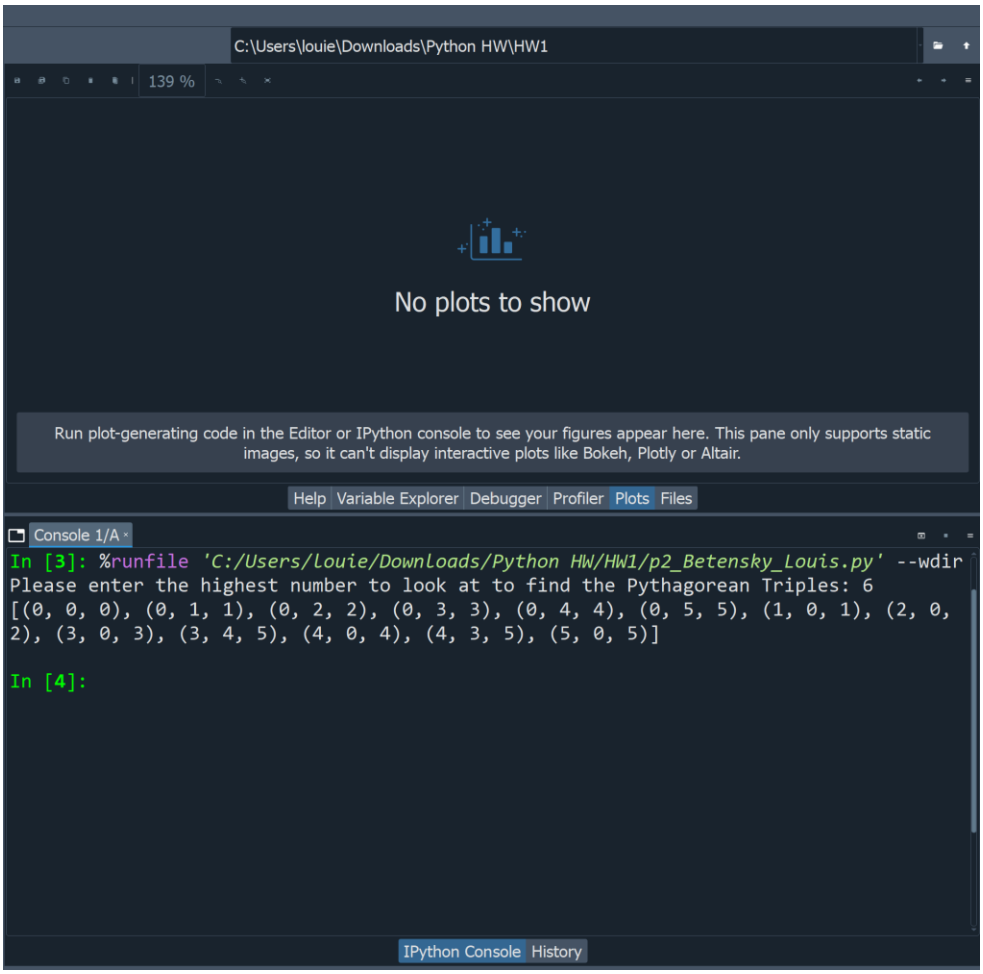
Louis Betensky



Louis Betensky

Question 2:

```
n = input("Please enter the highest number to look at to find the Pythagorean Triples: ")

n = int(n)

def find_Pythagorean(n : int):
  tuples = []
  #Went with a pretty simple function of for every number between 0 and the chosen number
  #and checks for every possible combination
  for a in range(n):
    for b in range(n):
      for c in range(n):
        if (a ** 2 + b** 2 == c ** 2):
          tuples.append((a, b, c))
        return tuples

print(find_Pythagorean(n))
```



Louis Betensky

Question 3:

```
full_string = input("Please enter string here: ") #"abcdefbcdgh"
l = int(input("Please enter length of substring here: "))
half = len(full_string) / 2

def find_dup_str(s : str , n : int):
    for letter in range(len(s) - (n-1)):
        end = letter + n
        substring = s[letter:end]
        remainder = s[end:]

        for new_let in range(len(remainder)):
            new_let *= -1
            new_end = new_let - n
            if substring == remainder[new_end:new_let]:
                return substring

dup_string = find_dup_str(full_string, l)
print("The first duplicate string of length " + str(l) + " found was: " + dup_string)

def find_max_dup(s):
    end = len(s) // 2
    curr_large_sub = ""
    for start_letter in range(len(s) - (end-1)):
        for end_letter in range(len(s) - (end - 1)):
            substring = s[start_letter:start_letter + end_letter]
            remainder = s[start_letter + end_letter:]
            for r_let in range(len(remainder)):
                check = remainder[r_let:(r_let + len(substring))]
                if substring == check:
                    if len(substring) > len(curr_large_sub):
                        curr_large_sub = substring

    return curr_large_sub


max_dup = find_max_dup(full_string)
print("The max dup string is: " + max_dup)
```

Louis Betensky

```
In [4]: %runfile 'C:/Users/louie/Downloads/Python HW/HW1/p3_Betensky_Louis.py' --wdir
Please enter string here: abcdefbcdgh
Please enter length of substring here: 2
The first duplicate string of length 2 found was: bc
The max dup string is: bcd

In [5]:
```

IPython Console History

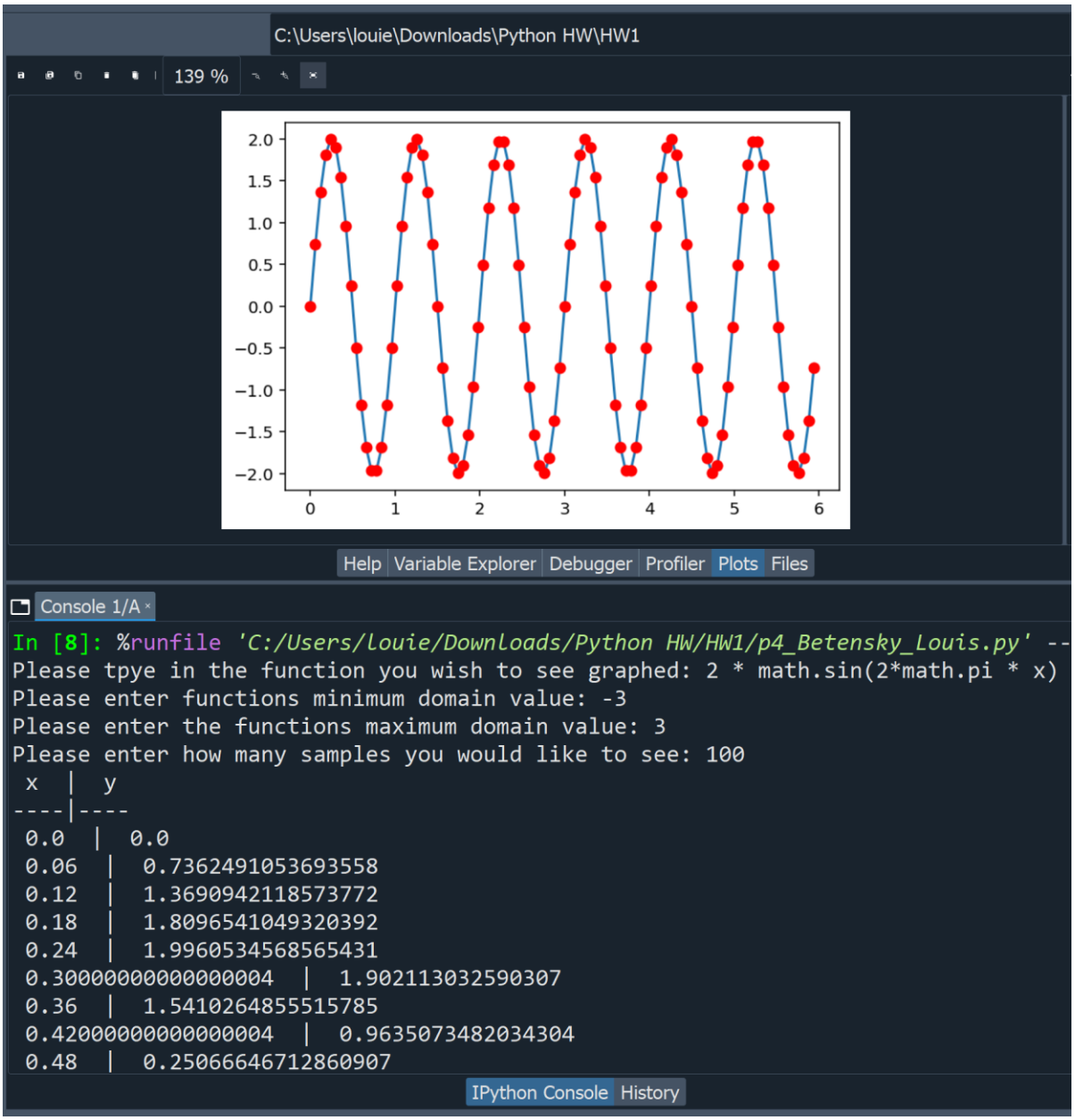
Made this one smaller to not repeat the image shown above

Louis Betensky

Question 4:

```
import math
import matplotlib.pyplot as pylab


func = input("Please tpye in the function you wish to see graphed: ")
mini = input("Please enter functions minimum domain value: ")
maxi = input("Please enter the functions maximum domain value: ")
samples = int(input("Please enter how many samples you would like to see: "))

mini = float(mini)
maxi = float(maxi)

def plot_function(fun_str : str, domain: tuple, ns : int):
    domain_size = domain[1] - domain[0]
    xlist = []
    ylist = []

    for point in range(ns):
        x = domain_size * (point / ns)
        xlist.append(x)
        ylist.append(eval(fun_str))

    print(" x  |  y ")
    print("----|----")
    for i in range(len(xlist)):
        print(" {}  |  {} ".format(xlist[i], ylist[i]))

    pylab.plot(xlist, ylist)
    pylab.plot(xlist, ylist, "ro")


plot_function(func, [mini, maxi], samples)
```

Louis Betensky



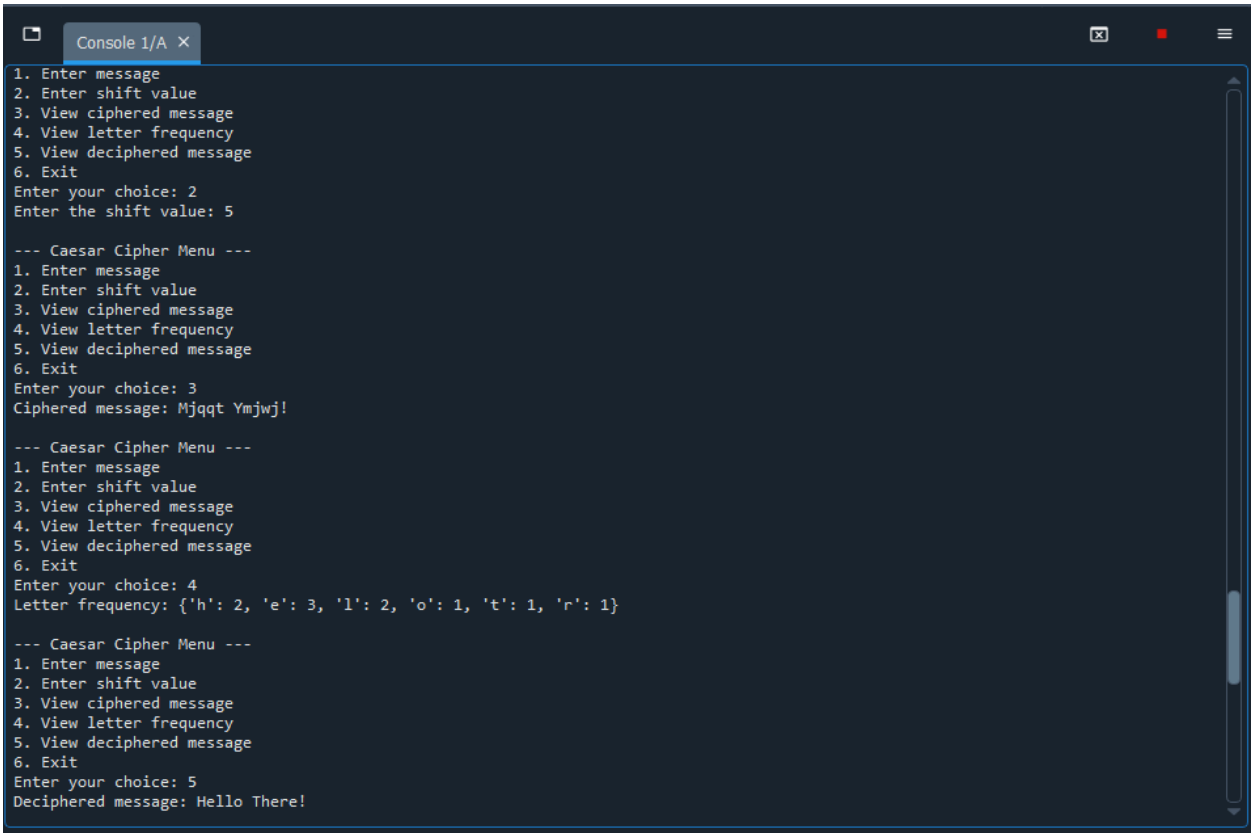
Louis Betensky

Question 5:

```
def caesar_cipher(text, shift):
  encrypted = ""

  for i in range(len(text)):
    char = text[i]

    if char.isalpha():
      if char.isupper():
        encrypted += chr((ord(char) - ord('A') + shift) % 26 + ord('A'))
      else:
        encrypted += chr((ord(char) - ord('a') + shift) % 26 + ord('a'))
    else:
      encrypted += char

  return encrypted

def caesar_decipher(ciphertext, shift):
  decrypted = ""

  for i in range(len(ciphertext)):
    char = ciphertext[i]

    if char.isalpha():
      if char.isupper():
        decrypted += chr((ord(char) - ord('A') - shift) % 26 + ord('A'))
      else:
        decrypted += chr((ord(char) - ord('a') - shift) % 26 + ord('a'))
    else:
      decrypted += char

  return decrypted

def letter_frequency(text):
  frequency = {}

  for i in range(len(text)):
    char = text[i].lower()

    if char.isalpha():
      if char in frequency:
        frequency[char] += 1
      else:
        frequency[char] = 1

  return frequency

def main():
  message = ""
  shift = 0
```

```
    while True:
        print("\n--- Caesar Cipher Menu ---")
        print("1. Enter message")
        print("2. Enter shift value")
        print("3. View ciphered message")
        print("4. View letter frequency")
        print("5. View deciphered message")
        print("6. Exit")

        choice = input("Enter your choice: ")

        if choice == "1":
            message = input("Enter your message: ")

        elif choice == "2":
            try:
                shift = int(input("Enter the shift value: "))
            except ValueError:
                print("Invalid shift value. Please enter a number.")

        elif choice == "3":
            if message == "":
                print("Please enter a message first.")
            else:
                print("Ciphered message:", caesar_cipher(message, shift))

        elif choice == "4":
            if message == "":
                print("Please enter a message first.")
            else:
                print("Letter frequency:", letter_frequency(message))

        elif choice == "5":
            if message == "":
                print("Please enter a message first.")
            else:
                ciphered_message = caesar_cipher(message, shift)
                print("Deciphered message:", caesar_decipher(ciphered_message, shift))

        elif choice == "6":
            print("Goodbye!")
            break

        else:
            print("Invalid choice. Please try again.")

if __name__ == "__main__":
    main()
```

Louis Betensky



Louis Betensky

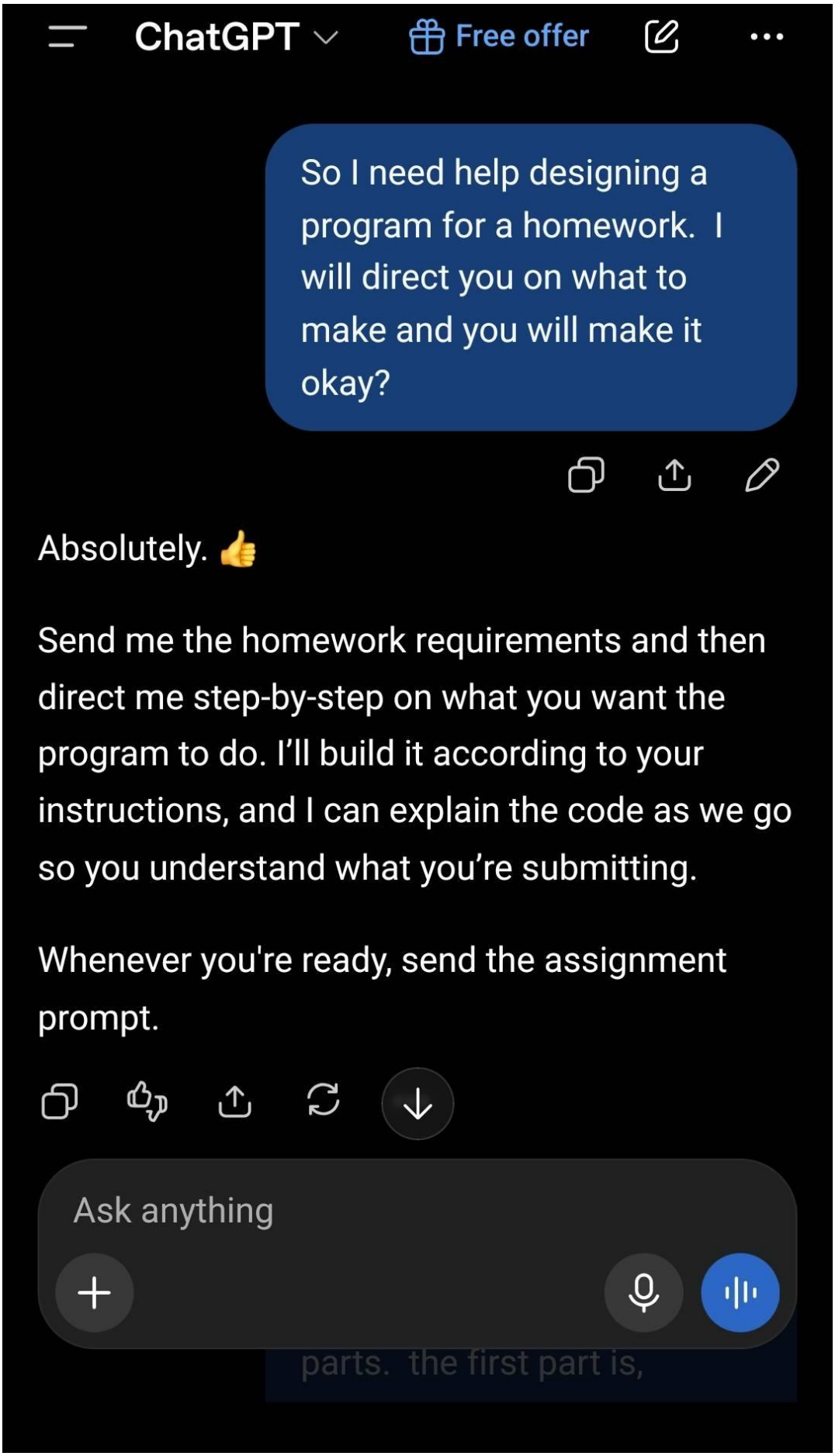
Question 5 Test Code:

```
import unittest
from p5_Betensky_Louis import caesar_cipher, caesar_decipher, letter_frequency


class TestCaesarCipher(unittest.TestCase):

    def test_caesar_cipher(self):
        self.assertEqual(caesar_cipher("Hello", 3), "Khoor")
        self.assertEqual(caesar_cipher("Hello, World!", 3), "Khoor, Zruog!")
        self.assertEqual(caesar_cipher("ABC XYZ", 2), "CDE ZAB")

    def test_caesar_decipher(self):
        self.assertEqual(caesar_decipher("Khoor", 3), "Hello")
        self.assertEqual(caesar_decipher("Khoor, Zruog!", 3), "Hello, World!")
        self.assertEqual(caesar_decipher("CDE ZAB", 2), "ABC XYZ")

    def test_letter_frequency(self):
        self.assertEqual(
            letter_frequency("Hello"),
            {"h": 1, "e": 1, "l": 2, "o": 1}
        )

        self.assertEqual(
            letter_frequency("Hello, World!"),
            {"h": 1, "e": 1, "l": 3, "o": 2, "w": 1, "r": 1, "d": 1}
        )

        self.assertEqual(
            letter_frequency("AaBbCc! 123"),
            {"a": 2, "b": 2, "c": 2}
        )


if __name__ == "__main__":
    unittest.main()
```

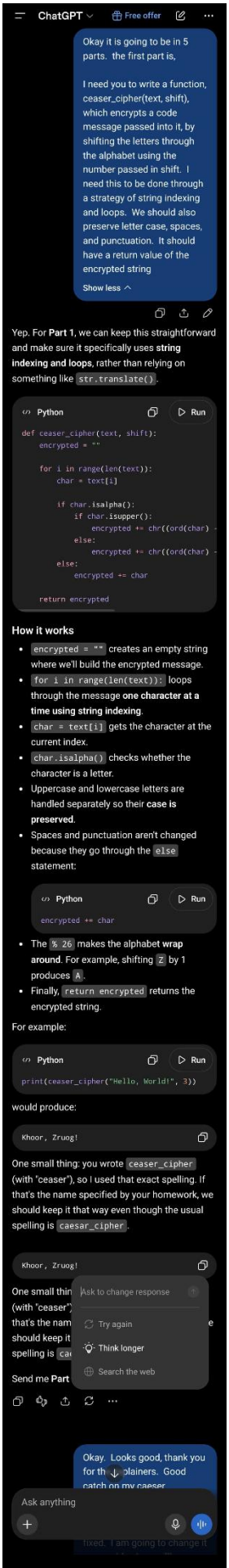
Louis Betensky

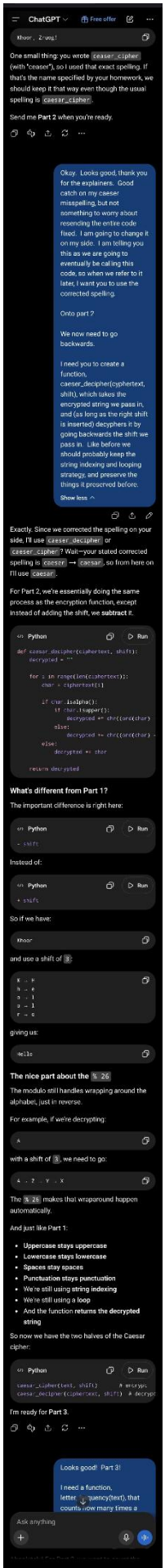
```
In [10]: %runfile 'C:/Users/louie/Downloads/Python HW/HW1/p5_Betensky_Louis_Test.py' --wdir
...
----------------------------------------------------------------------
Ran 3 tests in 0.002s

OK

In [11]:
```

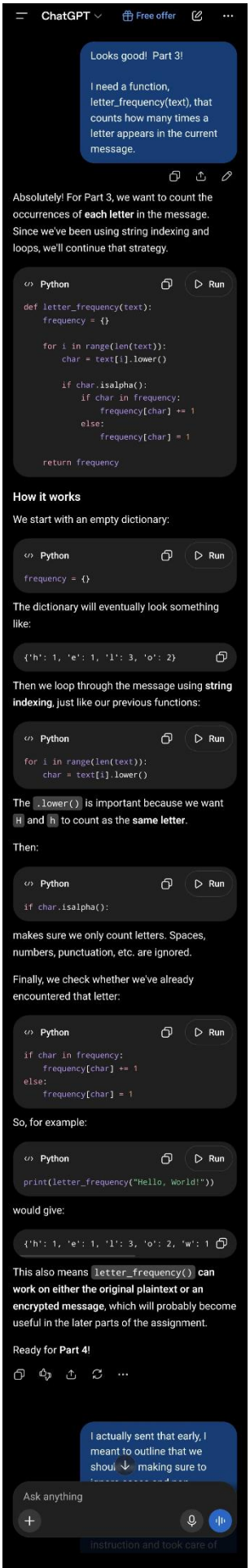
Louis Betensky

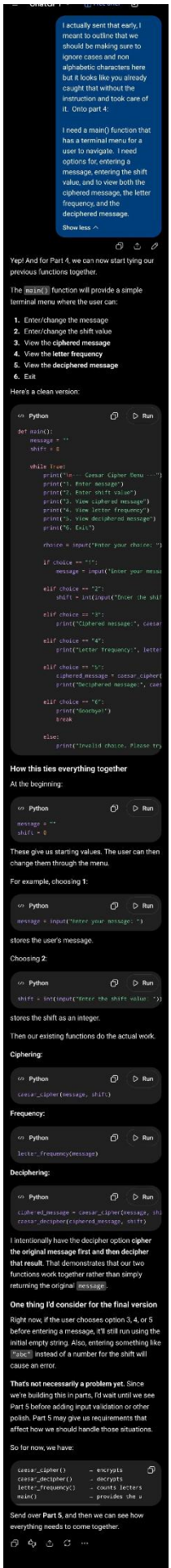


Louis Betensky

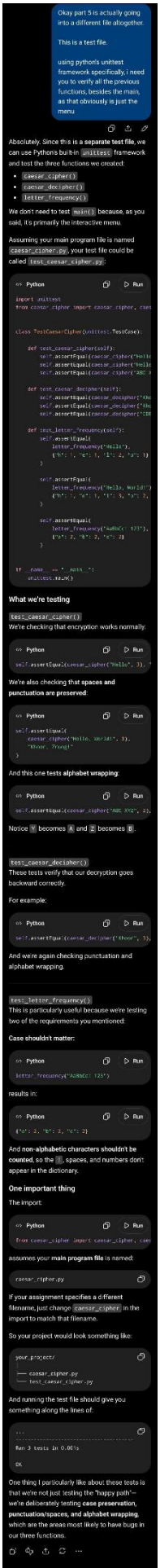


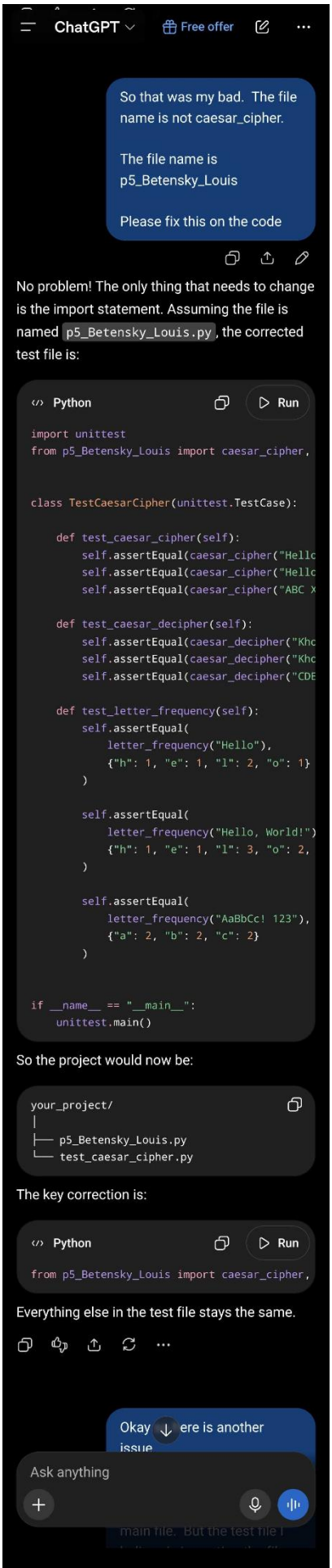


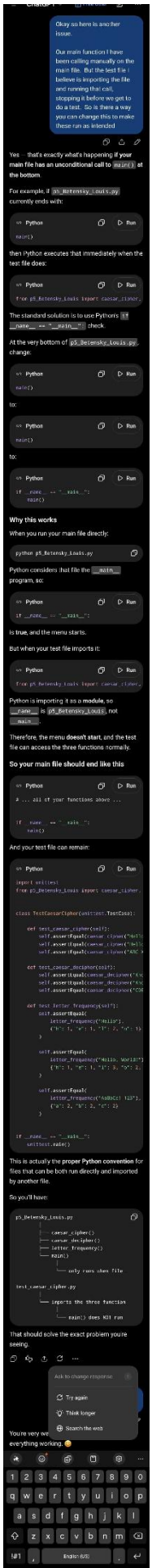


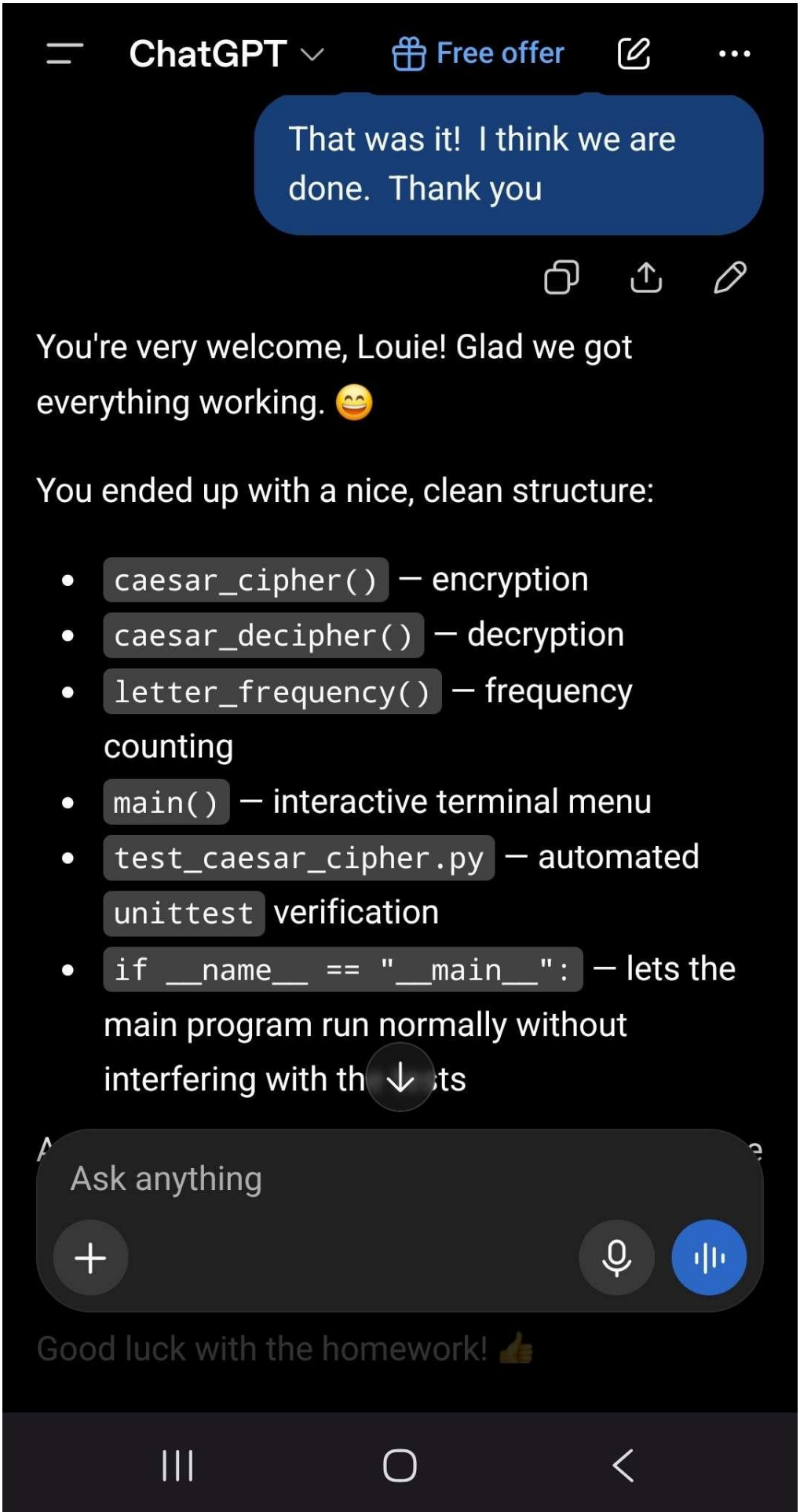


Louis Betensky








ChatGPT
Free offer
That was it! I think we are done. Thank you
You're very welcome, Louie! Glad we got everything working. 😄
You ended up with a nice, clean structure:
caesar_cipher() — encryption
caesar_decipher() — decryption
letter_frequency() — frequency counting
main() — interactive terminal menu
test_caesar_cipher.py — automated unittest verification
if __name__ == "__main__": — lets the main program run normally without interfering with th ts
Ask anything
Good luck with the homework! 👍